



UNIVERSITY *of* WEST FLORIDA

Machine Learning with Scikit-learn

Ensemble Learning

Brian Jalaian, Ph.D.

Associate Professor

Intelligent Systems & Robotics Department

Table of Contents

Ensemble Learning

Majority Voting

- Majority Voting - Ensemble error
- Simple Majority Vote Classifier

Bagging

Boosting

- AdaBoost
- Gradient Boosting
- XGBoost

Ensemble Learning Overview

Ensemble Learning

Overview

- By combining the predictions of several models, the ensemble can often outperform any single model in the ensemble.

Common techniques in ensemble learning

- *Majority voting*
 - We select the class label that has been predicted by the majority of classifiers, that is, received more than 50 percent of the votes. (for binary class settings)
- *Bagging*
 - It involves training multiple models in parallel, each on a random subset of the data.
- *Boosting*
 - Sequential training of models where each model tries to correct the errors of its predecessors.

Pros

- Improved accuracy; reduced overfitting; exploiting models' unique strengths

Cons

- Increased complexity; more computational cost; possible worse results

Majority Voting

Majority Voting

Overview

- Majority voting:
 - We select the class label that has been predicted by the majority of classifiers, that is, received more than 50 percent of the votes. (It's for binary class settings.)
- Plurality voting:
 - Same, but for multiclass settings.

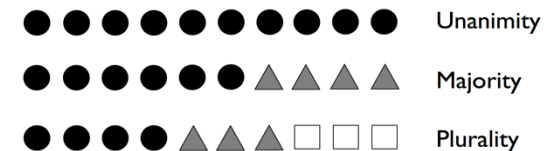


Figure 7.1: The different voting concepts

General ensemble approach via majority voting

- Using the training dataset, we start by training m different classifiers, $(C_1, \dots, C_m)$
- To predict a class label via simple majority or plurality voting:
 - combine the predicted class labels of each individual classifier, C_j
 - select the class label, $\hat{y}$, that received the most votes

$$\hat{y} = \text{mode}\{C_1(x), C_2(x), \dots, C_m(x)\}$$

(In statistics, the **mode** is the most frequent event or result in a set.

For example, **mode**{1, 2, 1, 1, 2, 4, 5, 4} = 1.)

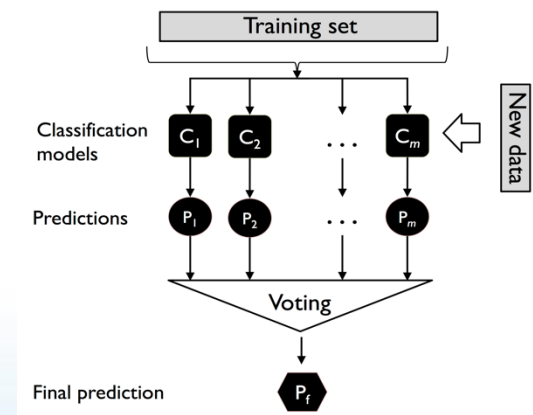


Figure 7.2: A general ensemble approach

Majority Voting - Ensemble error

Ensemble error

To prove why ensemble better than single classifier.

Assumptions

All n -base classifiers for a binary classification task have an equal *error rate*, ε .

The classifiers are independent, and the error rates are not correlated.

Error probability of an ensemble of base classifiers

Describe it as a probability mass function of a binomial distribution:

$$P(y \geq k) = \sum_k^n \binom{n}{k} \varepsilon^k (1 - \varepsilon)^{n-k} = \varepsilon_{\text{ensemble}}$$

- $\binom{n}{k} = \frac{n!}{(n-k)!k!} = \text{binomial coefficient : } n \text{ choose } k$

Majority Voting - Ensemble error

Ensemble error Example

- 11 base classifiers ($n=11$), where each classifier has an error rate of 0.25 ($\varepsilon=0.25$) :

$$P(y \geq k) = \sum_{k=6}^{11} \binom{11}{k} 0.25^k (1 - 0.25)^{11-k} = 0.034$$

Result

- Error rate of the ensemble (0.034) << Error rate of each individual classifier (0.25)
- When base error = 0.25, then ensemble error = 0.034 (shown in the plot)

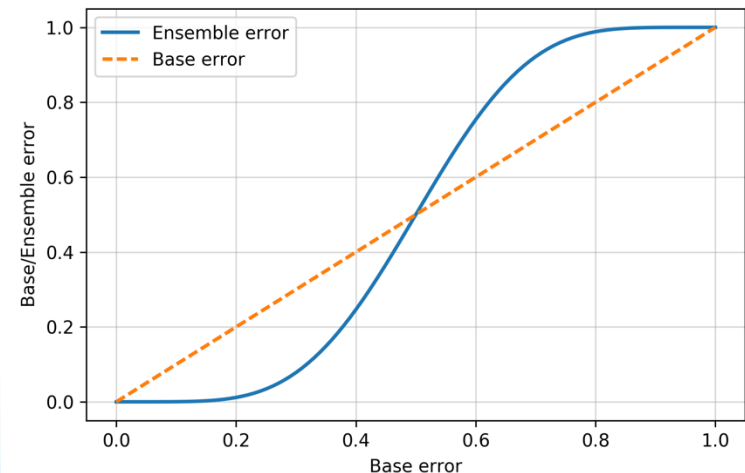


Figure 7.3: A plot of the ensemble error versus the base error

Simple Majority Vote Classifier



Goal

- To combine different classification algorithms associated with individual weights for confidence.
- Build a stronger meta-classifier that balances out the individual classifiers' weaknesses on a particular dataset.

The 2 different weighted majority votes

1. Weighted majority vote: class labels (2 cases)
 - 1) weight the predictions of each base classifier equally
 - 2) weight the predictions of each base classifier differently
2. Weighted majority vote: probabilities

Simple Majority Vote Classifier

General weighted majority vote

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j \chi_A(C_j(\mathbf{x}) = i)$$

- w_j : weight associated with a *base classifier*, C_j
- $\hat{y}$: predicted class label of the ensemble
- A : set of unique class labels
- χ_A : characteristic function or indicator function
 - returns 1 if the predicted class of the j -th classifier matches

For equal weights, simplify this equation:

$$\hat{y} = \text{mode}\{C_1(\mathbf{x}), C_2(\mathbf{x}), \dots, C_m(\mathbf{x})\}$$

Simple Majority Vote Classifier Example



Assumption

- An ensemble of three base classifiers, $C_j (j \in \{1,2,3\})$.

Goal

- Predict the class label, $C_j(x) \in \{0,1\}$, of a given example, x .

1) Weighted majority vote: class labels (equal weight)

$$C_1(x) \rightarrow 0, C_2(x) \rightarrow 0, C_3(x) \rightarrow 1$$
$$\hat{y} = \text{mode}\{0,0,1\} = 0$$

- Prediction: example belongs to class 0

2) Weighted majority vote: class labels (different weight)

- Assign a weight of 0.6 to C_3
- Assign a weight of 0.2 to C_1 and C_2

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j \chi_A(C_j(x) = i)$$
$$= \arg \max_i [0.2 \times i_0 + 0.2 \times i_0, 0.6 \times i_1] = 1$$

- Prediction: example belongs to class 1

Simple Majority Vote Classifier

Weighted majority vote: probabilities

Using the predicted class probabilities instead of the class labels for majority voting can be useful if the classifiers in our ensemble are well calibrated.

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j p_{ij}$$

- p_{ij} : predicted probability of the j -th classifier for class label i

Simple Majority Vote Classifier Example



Assumptions

- Assume that we have a binary classification problem with class labels, C_j , and an ensemble of three classifiers, $C_j (j \in \{1,2,3\})$.
- Assume that the classifiers C_j return the following class membership probabilities for a particular example, x :

$$C_1(x) \rightarrow [0.9, 0.1], C_2(x) \rightarrow [0.8, 0.2], C_3(x) \rightarrow [0.4, 0.6]$$

- (Here, binary class labels: $[i_0, i_1]$)

Calculate individual class probabilities

Using the same weights as previously (0.2, 0.2 and 0.6), we can then calculate the individual class probabilities as follows:

$$p(i_0 | x) = 0.2 \times 0.9 + 0.2 \times 0.8 + 0.6 \times 0.4 = 0.58$$

$$p(i_1 | x) = 0.2 \times 0.1 + 0.2 \times 0.2 + 0.6 \times 0.6 = 0.42$$

$$\hat{y} = \arg \max_i [p(i_0 | x), p(i_1 | x)] = 0$$

- Prediction: example belongs to class 0

Bagging

Bagging (= Bootstrap Aggregating)

Pros

- Improve the accuracy of unstable (complex, high data dimensional) models
- Decrease the degree of overfitting

Cons

- Ineffective in reducing model bias
 - Those models that are too simple to capture the trends in the data well.

Concept

Instead of using the same training dataset to fit the individual classifiers in the ensemble, we draw bootstrap samples (= random samples with replacement) from the initial training dataset.

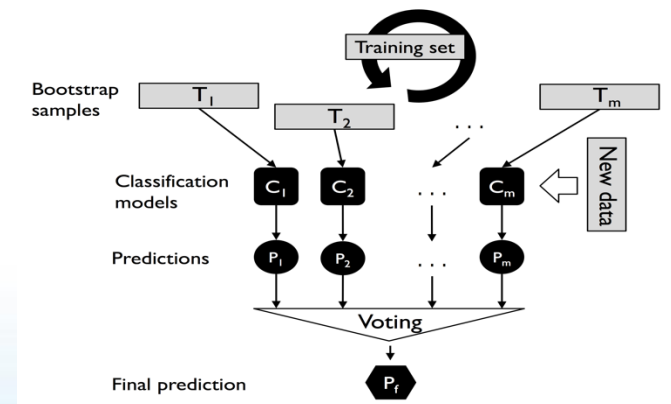


Figure 7.6: The concept of bagging

Bagging (= Bootstrap Aggregating)

Method Example

1. 7 training examples sampled randomly with replacement in each round of bagging.
2. Each bootstrap sample is then used to fit a classifier, C_j .
 - most typically an unpruned decision tree
3. Each classifier receives a random subset of examples from the training dataset.
4. Each subset contains a certain portion of duplicates and some of the original examples don't appear in a resampled dataset at all due to sampling with replacement.
5. Once the individual classifiers are fit to the bootstrap samples, the predictions are combined using majority voting.

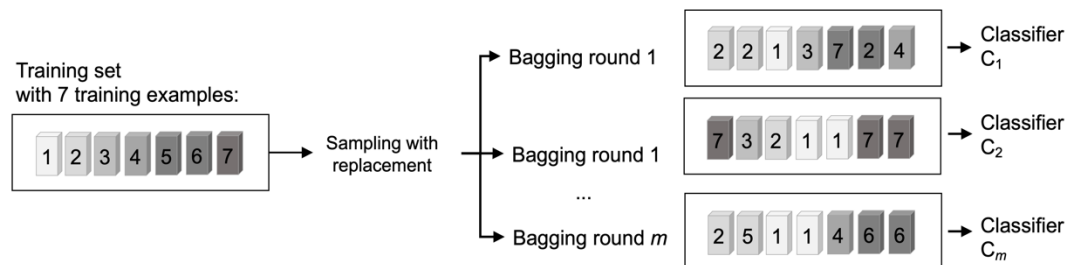


Figure 7.7: An example of bagging

Boosting (AdaBoost)

Boosting

Overview

- The key concept behind boosting is to focus on training examples that are hard to classify, that is, to let the **weak learners** (simple base classifier) subsequently learn from misclassified training examples to improve the performance of the ensemble.

Pros & Cons

- **Pros:** decrease bias and variance
- **Cons:** high variance overfitting

Method: Boosting

1. Draw a random subset (sample) of training examples, d_1 , without replacement from the training dataset, D , to train a weak learner, C_1 .
2. Draw a second random training subset, d_2 , without replacement from the training dataset and add 50 percent of the examples that were previously misclassified to train a weak learner, C_2 .
3. Find the training examples, d_3 , in the training dataset, D , which C_1 and C_2 disagree upon, to train a third weak learner, C_3 .
4. Combine the weak learners C_1 , C_2 , and C_3 via majority voting.

AdaBoost

Method: AdaBoost

AdaBoost uses the complete training dataset to train the weak learners, where the training examples are reweighted in each iteration to build a strong classifier that learns from the mistakes of the previous weak learners in the ensemble.

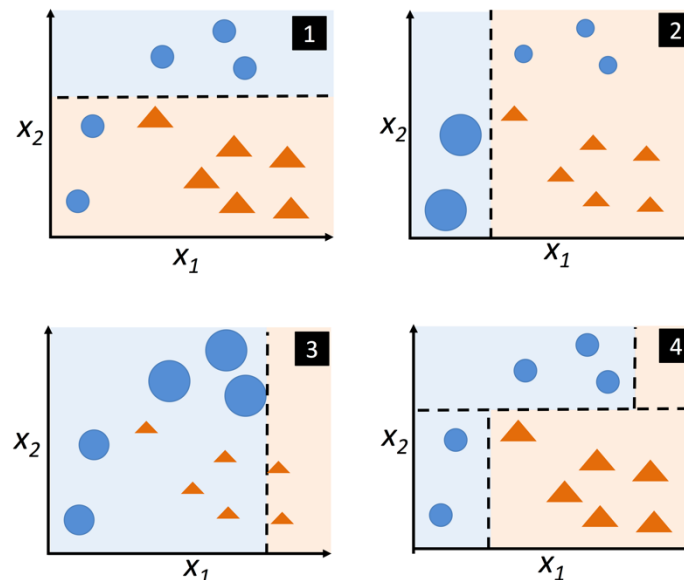


Figure 7.9: The concept of AdaBoost to improve weak learners

AdaBoost

Illustration: Subfigure 1

- A training dataset for binary classification where all training examples are assigned equal weights.
- Based on this training dataset, we train a decision stump, also known as single layer tree (dashed line) that tries to classify the examples of the two classes, and minimizing the loss function (or the impurity score in the special case of decision tree ensembles).

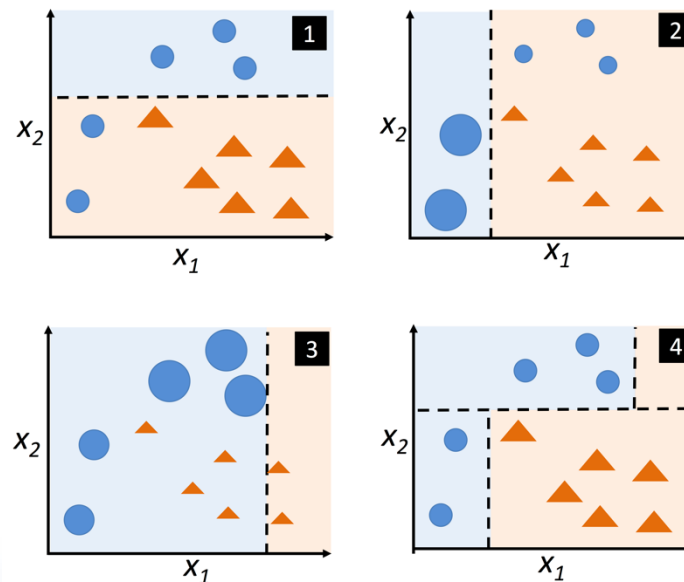


Figure 7.9: The concept of AdaBoost to improve weak learners

AdaBoost

Illustration: Subfigure 2

- Assign a larger weight to the two previously misclassified examples (big circles).
- Lower the weight of the correctly classified examples (small circles).
- The next decision stump will now be more focused on the training examples that have the largest weights—the training examples that are supposedly hard to classify.

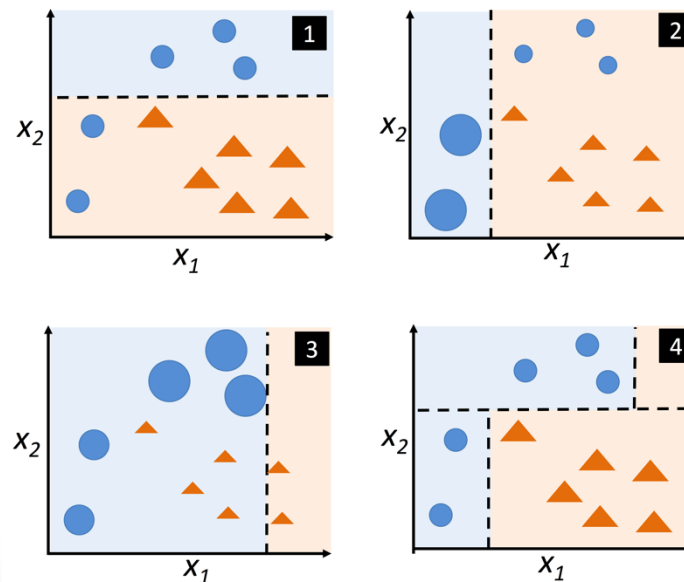


Figure 7.9: The concept of AdaBoost to improve weak learners

AdaBoost

Illustration: Subfigure 3

- The weak learner shown in subfigure 2 misclassifies three different examples from the circle class, which are then assigned a larger weight.

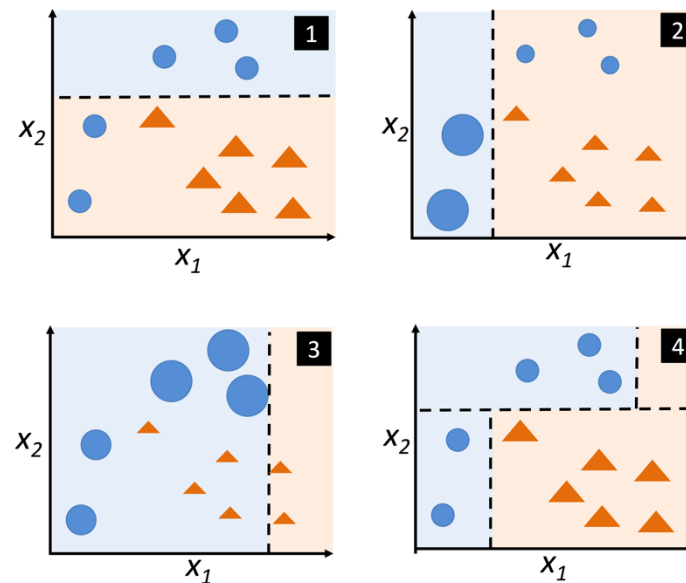


Figure 7.9: The concept of AdaBoost to improve weak learners

AdaBoost

Illustration: Subfigure 4

- Assuming that our AdaBoost ensemble only consists of three rounds of boosting, we then combine the three weak learners trained on different reweighted training subsets by a weighted majority vote.

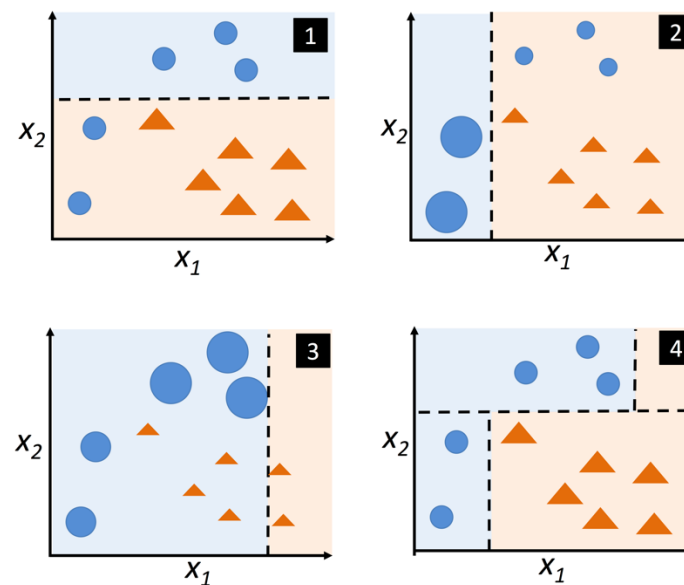


Figure 7.9: The concept of AdaBoost to improve weak learners

AdaBoost

AdaBoost algorithm

1. Set the weight vector, w , to uniform weights, where $\sum_i w_i = 1$.
2. For j in m boosting rounds, do the following:
 - a. Train a weighted weak learner: $C_j = \text{train}(X, y, w)$.
 - b. Predict class labels: $\hat{y} = \text{predict}(C_j, X)$.
 - c. Compute the weighted error rate: $\varepsilon = w \cdot (\hat{y} \neq y)$.
 - d. Compute the coefficient: $\alpha_j = 0.5 \log \frac{1-\varepsilon}{\varepsilon}$.
 - e. Update the weights: $w := w \times \exp(-\alpha_j \times \hat{y} \times y)$.
 - f. Normalize the weights to sum to 1: $w := w / \sum_i w_i$.
3. Compute the final prediction: $\hat{y} = \left(\sum_{j=1}^m \left(\alpha_j \times \text{predict}(C_j, X) \right) > 0 \right)$.

Note

- element-wise multiplication: cross symbol ($\times$)
- dot-product: dot symbol ($\cdot$)
- $(\hat{y} \neq y)$ in step 2 c refers to a binary vector consisting of 1 s and 0 s,
 - 1 is assigned, if the prediction is incorrect
 - 0 is assigned, otherwise

Gradient Boosting

Gradient Boosting

AdaBoost vs gradient boosting

Fundamentally, *gradient boosting* is very similar to *AdaBoost*.

- *AdaBoost* trains decision tree stumps based on errors of the previous decision tree stump.
 - The errors are used to compute sample weights in each round as well as for computing a classifier weight for each decision tree stump when combining the individual stumps into an ensemble.
 - Stop training once a maximum number of iterations (decision tree stumps) is reached.
- *Gradient boosting* fits decision trees in an iterative fashion using prediction errors.
 - Gradient boosting trees are usually deeper than decision tree stumps and have typically a maximum depth of 3 to 6 (or a maximum number of 8 to 64 leaf nodes).
 - The prediction errors are used directly to form the target variable for fitting the next tree.
 - It uses a global learning rate that is the same for each tree.

General Gradient Boosting Algorithm



Overview

In essence, gradient boosting builds a series of trees, where each tree is fit on the error (the difference between the label and the predicted value) of the previous tree. In each round, the tree ensemble improves as we are nudging each tree more in the right direction via small updates. These updates are based on a loss gradient, which is how gradient boosting got its name.

Algorithm overview

The algorithm contains 2 steps, and the 2nd step includes 4 sub steps (a – d).

General Gradient Boosting Algorithm

1. Initialize a model to return a *constant model prediction value*, $F_0(x)$.

- Use a decision tree root node (= a decision tree with a single leaf node) for $F_0(x)$.
- y : class label
- $\hat{y}$: predicted value returned by the tree (model)
 - Find this value by minimizing a differentiable loss function L .
- n : the n training examples in our dataset

$$F_0(x) = \arg \min_{\hat{y}} \sum_{i=1}^n L(y_i, \hat{y})$$

General Gradient Boosting Algorithm

2. For each tree $m = 1, \dots, M$, where M is a user-specified total number of trees:

2a. Compute the *pseudo-residual*, r_{im} .

- $r_{im} = \text{pseudo-residual} = \text{pseudo-response}$
- m : number of the tree
- $F(x) = \text{model prediction value}$: the prediction of the previous tree model, $F_{m-1}(x)$.
 - In the first round, this refers to $F_0(x)$ from the tree (single leaf node) from step 1.

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \dots, n$$

2b. Fit a tree to the *pseudo-residuals*, r_{im} .

General Gradient Boosting Algorithm

2c. For each leaf node, R_{jm} , we compute the following *output value*, γ_{jm} .

- $R_{jm} = \text{leaf node}$: the $j = 1 \dots J_m$ leaf nodes of the resulting tree in iteration m

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$

2d. Update the tree model by adding the *output value*, γ_m , to the previous tree.

- $\eta = \text{learning rate}$
- update model by taking small steps $\rightarrow$ avoid overfitting
- typically, a small value between 0.01 and 1

$$F_m(x) = F_{m-1}(x) + \eta \gamma_m$$

Gradient Boosting for Binary Classification



Mathematical definitions

For a single training example,

$$L_i = \text{logistic loss} = -y_i \log p_i + (1 - y_i) \log(1 - p_i)$$

$$\hat{y} = \text{predicted output} = \log(\text{odds}) = \log\left(\frac{p}{1 - p}\right)$$

We will use these $\log(\text{odds})$ to rewrite the logistic function as follows:

$$L_i = \log(1 + e^{\hat{y}_i}) - y_i \hat{y}_i$$

The derivative of this loss function with respect to the $\log(\text{odds})$:

$$\frac{\partial L_i}{\partial \hat{y}_i} = \frac{e^{\hat{y}_i}}{1 + e^{\hat{y}_i}} - y_i = p_i - y_i$$

Gradient Boosting for Binary Classification



Algorithm

Similar to the general form but reformulate them for binary classification scenario.

1. Create a root node that minimizes the logistic loss.

- The loss is minimized if the root node returns the $\log(odds) = \hat{y}$.

2. For each tree $m = 1, \dots, M$, where M is a user-specified number of total trees :

2a. We compute the *pseudo-residual*, r_{im} .

- Convert the $\log(odds)$ into a probability :

$$p = \frac{1}{1 + e^{-\hat{y}}}$$

- r_{im} : difference between the class label and the predicted probability:

$$r_{im} = -\frac{\partial L_i}{\partial \hat{y}_i} = y_i - p_i, \text{ for } i = 1, \dots, n$$

2b. Fit a new tree to the pseudo-residuals.

Gradient Boosting for Binary Classification



2c. For each leaf node, R_{jm} , compute a value γ_{jm} that minimizes the logistic loss function.

- This includes a summarization step for dealing with leaf nodes that contain multiple training examples:

$$\begin{aligned}\gamma_{jm} &= \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma) \\ &= \log(1 + e^{\widehat{y}_i + \gamma}) - y_i(\widehat{y}_i + \gamma) \\ &= \frac{\sum_i y_i - p_i}{\sum_i p_i(1 - p_i)}\end{aligned}$$

- Note that the summation here is only over the examples at the node corresponding to the leaf node R_{jm} and not the complete training set.

2d. Update the model by adding γ_{jm} from step 2c with learning rate η :

$$F_m(x) = F_{m-1}(x) + \eta \gamma_m$$

Gradient Boosting for Binary Classification

Example

Let's apply it to a small toy example, a training dataset of the following three examples shown as follow.

	Feature x_1	Feature x_2	Class label y
1	1.12	1.4	1
2	2.45	2.1	0
3	3.54	1.2	1

Figure 7.12: Toy dataset for explaining gradient boosting

Step 1

- Let's start with step 1, constructing the root node and computing the $\log(odds)$.

Gradient Boosting for Binary Classification

Step 2a

- Convert the *log (odds)* into *class-membership probabilities*, p .
- Compute the *pseudo-residuals*, r .

The odds can be computed as the number of successes divided by the number of failures. Here, we regard label 1 as success and label 0 as failure, so the odds are computed as: $odd = 2/1$.

	Feature x_1	Feature x_2	Class label y	Step 1: $\hat{y} = \log(\text{odds})$	Step 2A: $p = \frac{1}{1 + e^{-\hat{y}}}$	Step 2A: $r = y - p$
1	1.12	1.4	1	0.69	0.67	0.33
2	2.45	2.1	0	0.69	0.67	-0.67
3	3.54	1.2	1	0.69	0.67	0.33

Figure 7.13: Results from the first round of applying step 1 and step 2a

Gradient Boosting for Binary Classification

Step 2b

- Fit a new tree on the *pseudo-residuals*, r .

Step 2c

- Compute the *output values*, γ , for this tree.

Note that we artificially limit the tree to have only two leaf nodes, which helps illustrate what happens if a leaf node contains more than one example.

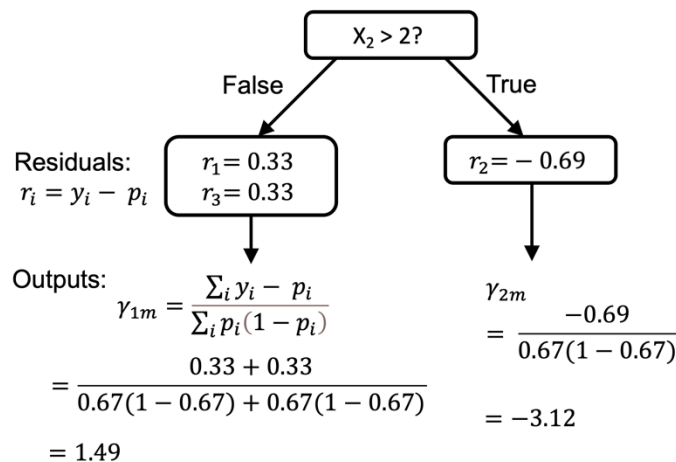


Figure 7.14: An illustration of steps 2b and 2c, which fits a tree to the residuals and computes the output values for each leaf node

Gradient Boosting for Binary Classification

Step 2d

- Update the previous model and the current model.
- Assume a learning rate of $\eta = 0.1$, the resulting prediction for the first training example.



	Feature x_1	Feature x_2	Class label y
1	1.12	1.4	1
2	2.45	2.1	0
3	3.54	1.2	1

Step 2D: $F_m(x) = F_{m-1}(x) + \eta \gamma_m$

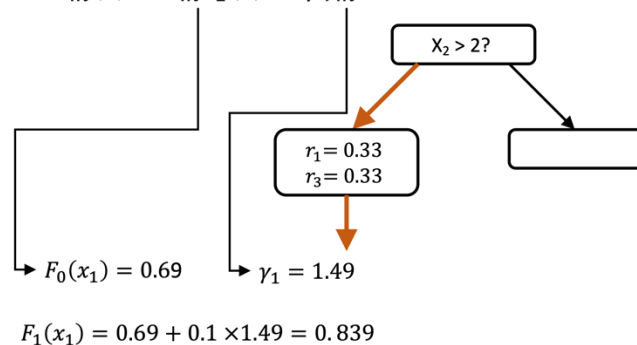


Figure 7.15: The update of the previous model shown in the context of the first training example

Gradient Boosting for Binary Classification

2nd round of steps 2a to 2d ($m = 2$)

In the second round, we use the $\log(odds)$ returned by the updated model, for example, $F_1(x_1) = 0.839$, as input to step 2a. The new values we obtained in the second round are shown in the following figure.

x_1	x_2	y	Step 1: $F_0(x) = \hat{y}$ $= \log(\text{odds})$	Step 2A: $p = \frac{1}{1 + e^{-\hat{y}}}$	Step 2A: $r = y - p$	New $\log(\text{odds})$ $\hat{y} = F_1(x)$	Step 2A: p	Step 2A: r	
1	1.12	1.4	1	0.69	0.67	0.33	0.839	0.698	0.302
2	2.45	2.1	0	0.69	0.67	-0.67	0.378	0.593	-0.593
3	3.54	1.2	1	0.69	0.67	0.33	0.839	0.698	0.302

Round $m = 1$

Round $m = 2$

Figure 7.16: Values from the second round next to the values from the first round

We can already see that the *predicted probabilities*, p , are higher for the positive class and lower for the negative class. Consequently, the residuals are getting smaller, too.

Note that the process of steps 2a to 2d is repeated until we have fit M trees or the residuals are smaller than a user-specified threshold value. Then, once the gradient boosting algorithm has completed, we can use it to predict the class labels by thresholding the probability values of the final model, $F_M(x)$ at 0.5.

XGBoost (Extreme Gradient Boosting)



Overview

1. Gradient Boosting Framework

- XGBoost is an implementation of gradient boosted decision trees designed for speed and performance.

2. Efficiency at Scale

- It is optimized for both memory efficiency and computational speed, making it suitable for large datasets.

3. Regularization

- Includes L1 (Lasso regression) and L2 (Ridge regression) regularization to prevent overfitting, which is not standard in all gradient boosting frameworks.

4. Handling of Sparse Data

- Efficiently processes sparse data (missing data or zero values), which is common in many real-world datasets.

XGBoost (Extreme Gradient Boosting)



Pros

1. **High Performance**
 - Often provides better predictive accuracy than other machine learning algorithms.
2. **Versatility**
 - Can be used for various types of data and regression, classification, ranking, and user-defined prediction problems.
3. **Flexibility**
 - Supports various loss functions and is easily customizable.
4. **Cross-platform**
 - Works on Linux, Windows, and macOS, and it can be integrated with several programming languages, including Python, R, and Java.

Cons

1. **Complexity**
 - The complexity of the model can make it difficult to interpret.
2. **Computational Resources**
 - While efficient, training large models on very large datasets can be resource-intensive.
3. **Hyperparameter Tuning**
 - Requires careful tuning of parameters to achieve the best performance.